

```

const CORS={"Access-Control-Allow-Origin":"*","Access-Control-Allow-Methods":"GET
const START_CASH=1000000;
const CANDLE_MS=60000; // 1-minute candles
const MAX_CANDLES=500;
const I={
  RELIANCE:{name:"Reliance Industries",sector:"Stocks",price:2925.4,lot:1},TCS:{nam
  HDFCBANK:{name:"HDFC Bank",sector:"Banking",price:1948.6,lot:1},INFY:{name:"Infos
  ICICIBANK:{name:"ICICI Bank",sector:"Banking",price:1425.7,lot:1},SBIN:{name:"Sta
  BHARTIARTL:{name:"Bharti Airtel",sector:"Telecom",price:1810.5,lot:1},ITC:{name:"
  LT:{name:"Larsen & Toubro",sector:"Industrials",price:3840.8,lot:1},MARUTI:{name:
  TATASTEEL:{name:"Tata Steel",sector:"Metals",price:178.4,lot:10},SUNPHARMA:{name:
  HINDUNILVR:{name:"Hindustan Unilever",sector:"FMCG",price:2765.1,lot:1},AXISBANK:
  NIFTY50:{name:"NIFTY 50",sector:"Index",price:25840.2,lot:1},BANKNIFTY:{name:"NIF
  GOLD:{name:"Gold (educational proxy)",sector:"Commodity",price:112850,lot:1},SILV
  DIAMOND:{name:"Diamond (educational proxy)",sector:"Educational",price:100000,lot
const json=(x,s=200)=>new Response(JSON.stringify(x),{status:s,headers:{"content-

function freshCandles(){
  let c={};
  for(let k in I) c[k]=[{t:Math.floor(Date.now()/CANDLE_MS)*CANDLE_MS,o:I[k].pric
  return c;
}
const seed=()=>({
  cash:START_CASH,
  positions:{},
  orders:[],
  prices:Object.fromEntries(Object.entries(I).map(([k,v])=>[k,v.price])),
  dayOpen:Object.fromEntries(Object.entries(I).map(([k,v])=>[k,v.price])),
  dayHigh:Object.fromEntries(Object.entries(I).map(([k,v])=>[k,v.price])),
  dayLow:Object.fromEntries(Object.entries(I).map(([k,v])=>[k,v.price])),
  candles:freshCandles(),
  updatedAt>Date.now()
});

function open(){let d=new Date(new Date().toLocaleString("en-US",{timeZone:"Asia/

function tickOne(s,k){
  let old=s.prices[k];
  let np=Math.max(.01, old*(1+(Math.random()-.5)*.0016));
  s.prices[k]=np;
  if(np>s.dayHigh[k])s.dayHigh[k]=np;
  if(np<s.dayLow[k])s.dayLow[k]=np;
  let vol=Math.round(Math.abs(np-old)/old*5000000+Math.random()*8000);

```

```

let bucket=Math.floor(Date.now()/CANDLE_MS)*CANDLE_MS;
let arr=s.candles[k];
let last=arr[arr.length-1];
if(last&&last.t===bucket){
    last.h=Math.max(last.h,np); last.l=Math.min(last.l,np); last.c=np; last.v+=vo
} else {
    arr.push({t:bucket,o:old,h:Math.max(old,np),l:Math.min(old,np),c:np,v:vol});
    if(arr.length>MAX_CANDLES) arr.shift();
}
}

function advance(s){
    if(!s.candles) return Object.assign(s,seed(),{cash:s.cash,positions:s.positions
    let n=Date.now(),steps=Math.min(40,Math.floor(Math.max(0,n-s.updatedAt)/2500));
    if(open()) for(let k=0;k<steps;k++) for(let sym in I) tickOne(s,sym);
    s.updatedAt=n;
    return s;
}

export class MarketState{
    constructor(state){this.state=state}
    async get(){let s=await this.state.storage.get("state"); return s&&s.candles?s:
    async put(s){await this.state.storage.put("state",s);return s}
    async fetch(req){
        let s=advance(await this.get()),u=new URL(req.url);

        if(u.pathname==="/api/state"&&req.method==="GET"){
            await this.put(s);
            let change={};
            for(let k in I) change[k]=+(((s.prices[k]-s.dayOpen[k])/s.dayOpen[k])*100).
            return json({ok:true,onlineOnly:true,serverTime:new Date().toISOString(),ma
        }

        if(u.pathname==="/api/candles"&&req.method==="GET"){
            let sym=String(u.searchParams.get("symbol")|| "").toUpperCase();
            if(!I[sym]) return json({ok:false,error:"Unknown instrument"},400);
            await this.put(s);
            return json({ok:true,symbol:sym,candles:s.candles[sym]|| []});
        }

        if(u.pathname==="/api/order"&&req.method==="POST"){
            let b; try{b=await req.json()}catch{return json({ok:false,error:"Invalid JS
            let sym=String(b.symbol|| "").toUpperCase(),side=String(b.side|| "").toUpperCase
            if(!x)return json({ok:false,error:"Unknown instrument"},400);
            if(!["BUY","SELL"].includes(side)||!Number.isInteger(q)||q<=0||q%x.lot)retu
            s=advance(s);

```

```

    let p=s.prices[sym],value=p*q,fee=value*.0005,pos=s.positions[sym]||{qty:0,
    if(side==="BUY"){
        if(s.cash<value+fee)return json({ok:false,error:"Insufficient virtual cas
        pos.avg=(pos.avg*pos.qty+value)/(pos.qty+q); pos.qty+=q; s.cash-=value+fe
    } else {
        if(pos.qty<q)return json({ok:false,error:"Insufficient virtual units"},40
        pos.qty-=q; s.cash+=value-fee; if(!pos.qty)delete s.positions[sym];
    }
    if(pos.qty)s.positions[sym]=pos;
    let o={id:crypto.randomUUID(),symbol:sym,side,qty:q,price:p,value,brokerage
    s.orders.push(o); await this.put(s);
    return json({ok:true,order:o,cash:s.cash,positions:s.positions});
}

if(u.pathname==="/api/reset"&&req.method==="POST"){
    let fresh=seed(); await this.put(fresh);
    return json({ok:true,reset:true});
}

return json({ok:false,error:"Not found"},404);
}
}

export default{
    async fetch(req,env){
        if(req.method==="OPTIONS")return new Response(null,{headers:CORS});
        let url=new URL(req.url);
        if(url.pathname.startsWith("/api/")){
            let uid=url.searchParams.get("uid")||"guest";
            let id=env.MARKET_STATE.idFromName("player-"+uid);
            return env.MARKET_STATE.get(id).fetch(req);
        }
        return new Response("MarketIQ API server is running.",{headers:CORS});
    }
}

```